

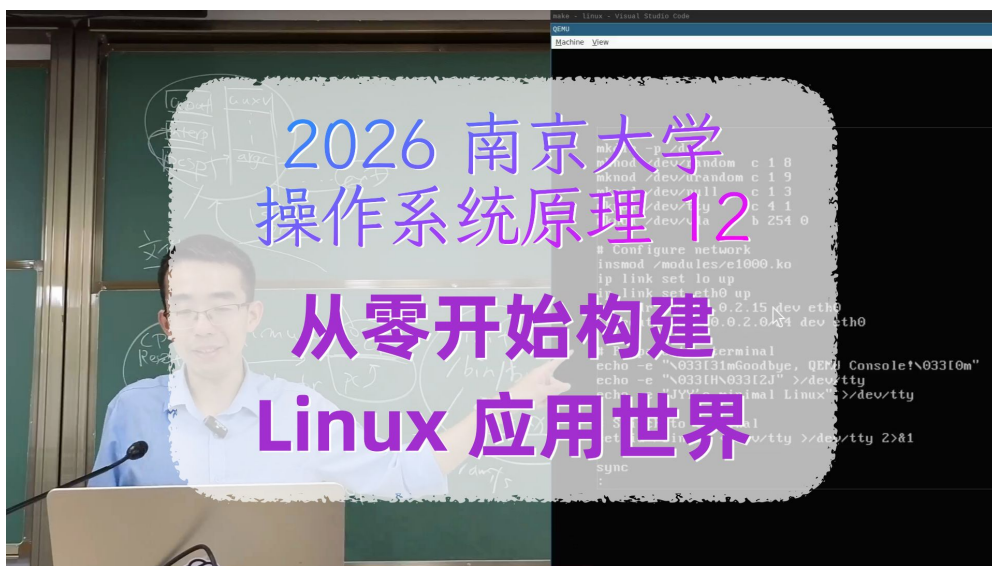
课程笔记

从零开始构建 Linux 应用世界

2026 南京大学《操作系统原理》第 12 讲

lecture-to-notes

2026 年 4 月 18 日



视频作者/频道：绿导师原谅你了（蒋炎岩 / jyy）

发布日期：2026-04-09

视频时长：1:26:22

视频链接：<https://www.bilibili.com/video/BV17JDWBsEMi/>

目录

1 本课定位：虚拟化部分的收官之作	2
1.1 Virtualization 的四块拼图	2
1.2 今天要讲的三件事	3
1.3 本章小结	3
2 从 UNIX 到 Linux：操作系统的历史演化	4
2.1 起点：《The Evolution of the Unix Time-Sharing System》	4
2.1.1 27 行汇编的 fork	4
2.2 MINIX：Tanenbaum 的启蒙之作	4
2.2.1 Live demo：在树莓派上跑 MINIX 1.7	5
2.2.2 微内核 vs 宏内核：一个过早的正确想法	5
2.3 Linus Torvalds：just a hobby (1991-08-25)	6
2.3.1 时代的错位：Rosenblatt 的 perceptron	6
2.4 Tanenbaum-Torvalds 大论战	7
2.4.1 名场面：Linus vs NVIDIA (2012)	8
2.5 中场插曲：AI 时代的学习方式	8
2.6 本章小结	8
3 从 initramfs 开始构建 Linux 世界	10
3.1 起点：所有系统的”初始状态”	10
3.1.1 启动链的全景	10
3.2 initramfs：第一个进程住在哪里？	11
3.3 动手：最小的 init	11
3.3.1 第一步：一个只会 write+exit 的 x86-64 静态 ELF	11
3.3.2 第二步：把它打包进 initramfs	12
3.3.3 第三步：见证 kernel panic	13
3.4 Linux 真正的启动全流程：initramfs + pivot_root	13
3.4.1 解剖真机 initrd	13
3.4.2 pivot_root：从 initramfs 换到真实根	14
3.4.3 一份自制的完整 init 脚本	15
3.5 pivot_root 之后：一个真正可用的 Linux	16
3.6 从系统调用到应用生态	17
3.7 本章小结	17
4 总结与延伸	19
4.1 讲者核心观点	19
4.2 可操作的知识蒸馏	19
4.3 跨课程的 synthesis	19
4.4 拓展阅读	20

1 本课定位：虚拟化部分的收官之作

本讲（第 12 讲）是 2026 南大操作系统课「虚拟化」部分的最后一次课。jyy 把它定位成一次 *Hacking Day*——不是新的考点，而是把整学期学过的系统调用串成一条完整的故事线：从 CPU 一上电、到操作系统加载的**第一个进程**、再到被我们习以为常的**整个 Linux 应用世界**，是怎样一层层长出来的。

贯穿整个虚拟化部分的 single message

这半学期所有的课——进程与地址空间、文件描述符、C 标准库、链接与加载——看似话题各异，其实一直在回答同一个问题：**操作系统给应用程序提供了什么样的 API**。每一个被反复提到的 `fork`、`mmap`、`execve`、`open` 都不是魔法，都是教科书里可以逐字节展开的系统调用。

1.1 Virtualization 的四块拼图

讲者在开头用 5 分钟对已经讲过的内容做了一次系统性串讲：

- **进程与地址空间**：从「游戏修改器」视角切入，理解进程地址空间不是抽象符号，而是被硬件 MMU 翻译的具体页表。
- **文件描述符**：打开操作系统里的各种对象（文件、管道、socket、pidfd）统一成一把「句柄」，PS3、sperm 实验就是把这个概念做实。
- **C 标准库**：从 `printf` 一层层拨开，看到 glibc 如何把系统调用封装成熟悉的 API。
- **链接与加载**：上一讲的「Funny Little Executable」合上了虚拟化部分最后一片拼图——理解了 `execve` 到底做了什么，就理解了一个**进程从无到有的全部过程**。

execve 的完整行为一句话复述

`execve(pathname, argv, envp)` 的「确定性行为」：把 `pathname` 对应的 ELF 文件里所有 PT_LOAD 段搬到新地址空间，准备好 Stack Pointer 指向的初始栈（含 `argv`、`envp`、`auxv`），并让 PC 指向 interpreter（或静态连接程序自身）的第一条指令。之后的**所有事情**——动态库 `mmap`、符号解析、`_start`→`main`——都是那一条 PC 指令开始的用户态代码。

1.2 今天要讲的四件事

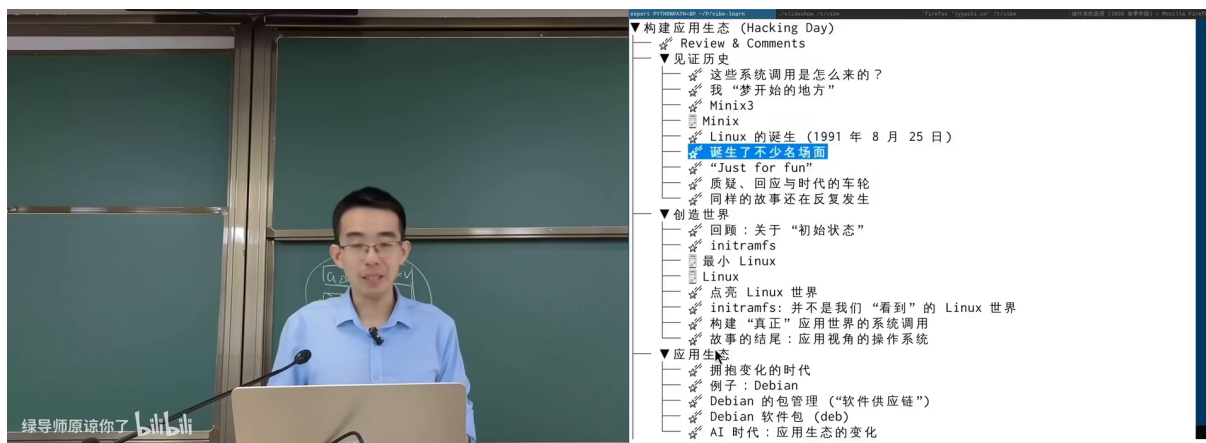


图 1: 本讲 Hacking Day 的完整 outline: 历史 (UNIX→MINIX→Linux)、动手 (从 initramfs 构建最小 Linux)、生态 (应用与包管理)¹

按讲者自己的排布：

1. **历史线**：从 Dennis Ritchie 的《The Evolution of the Unix Time-Sharing System》讲起，到 MINIX、再到 Linus 在 1991 年的「just a hobby」帖子。
2. **动手线**：亲手把一个**最小二进制文件**塞进 initramfs，让它成为 Linux 加载的第一个进程；再逐步扩展到 BusyBox、驱动、pivot_root、直至跑起一个可以**外部浏览器访问**的 httpd。
3. **生态线**：从狭义 (OS API) 到广义 (核心工具链、库、包管理)，解释「为什么中国没有国产操作系统」这个常见问题真正指向的是生态，不是内核。

1.3 本章小结

本讲不引入新的考点，但给出一张**完整的地图**：前面讲的每一个系统调用都是这张地图上的一个格子，今天要做的就是把所有格子连起来，证明「一个真实 Linux 从电源到浏览器打开抖音」的全链路**没有任何魔法**，都可以沿系统调用边界走一遍。

¹ 视频画面时间区间：00:28:03–00:28:18。

2 从 UNIX 到 Linux：操作系统的历史演化

2.1 起点：《The Evolution of the Unix Time-Sharing System》

jyy 反复推荐 Dennis Ritchie 1984 年这篇回忆性论文，因为它给出了一个非常干净的「从零开始」的视角：最早的 UNIX 甚至没有 `fork`。

最早 UNIX 的进程模型

最初的 UNIX shell 不能开子进程。要运行 `ls`，shell 的做法是：

1. 关闭所有打开的文件；
2. 将 `fd 0` 和 `fd 1` 重新打开到终端；
3. 把自己从内存卸载 (!)；
4. 通过早期的 `execve`（「加载器」）把 `ls` 的二进制搬到同一片内存并执行；
5. `ls` 退出后，shell 再被重新加载回来。

也就是说同一台终端任意时刻只有一个用户态程序——不是 shell，就是当前命令。

2.1.1 27 行汇编的 `fork`

这个限制后来被一个非常小的改动打破：Ken Thompson 用 27 行汇编在 PDP-7 上实现了 `fork`。原因其实非常物理——早期 UNIX 的地址空间抽象就是几个指针（代码段首/尾、数据段首/尾、栈段首/尾），复制一个进程就相当于复制几组指针和几个段。

一个反复出现的启发

「所有复杂的东西，在最开始都是很简单的。」今天看 Linux kernel 几千万行代码、大模型 serving 里的 PD 分离 / KV cache 之类复杂技术，都不要忘了它们的起点。同理 GPT-2 推理——本课后半程的并发实验，也会让你亲眼看到一份能跑起 GPT-2 的普通 C 代码。

2.2 MINIX：Tanenbaum 的启蒙之作

1987 年，阿姆斯特丹自由大学教授 Andrew S. Tanenbaum 写了一个叫 MINIX 的教学 OS（配套书《Operating Systems: Design and Implementation》）。

- MINIX 1 (1987)：UNIX v7 兼容。
- MINIX 2 (1997)：POSIX 兼容。
- MINIX 3 (2006)：今日仍在维护的成熟系统，许可证非常宽松。

一个意外的冷知识：世界上最流行的操作系统是 MINIX ？

Intel Management Engine (ME) 是每一颗 Intel CPU 芯片组上的独立协处理器，它需要运行一个完整操作系统，而这个系统正是 MINIX 3——因为它许可证宽松、体积小、稳定。发现这一点之后，有人打趣：在 Intel 如日中天的年代，每一台 Windows/Linux 电脑里其实都藏着一个 MINIX。

2.2.1 Live demo: 在树莓派上跑 MINIX 1.7

课上作者在 ARM 架构的树莓派上通过 QEMU 模拟 x86，用一条 `make minix2` 就生成了 64 MB 的 MINIX 1.7/2.0 磁盘镜像，并启动到一个完整的 UNIX v7 兼容环境——内核日志明确打印 MINIX: running in 16-bit protected mode。

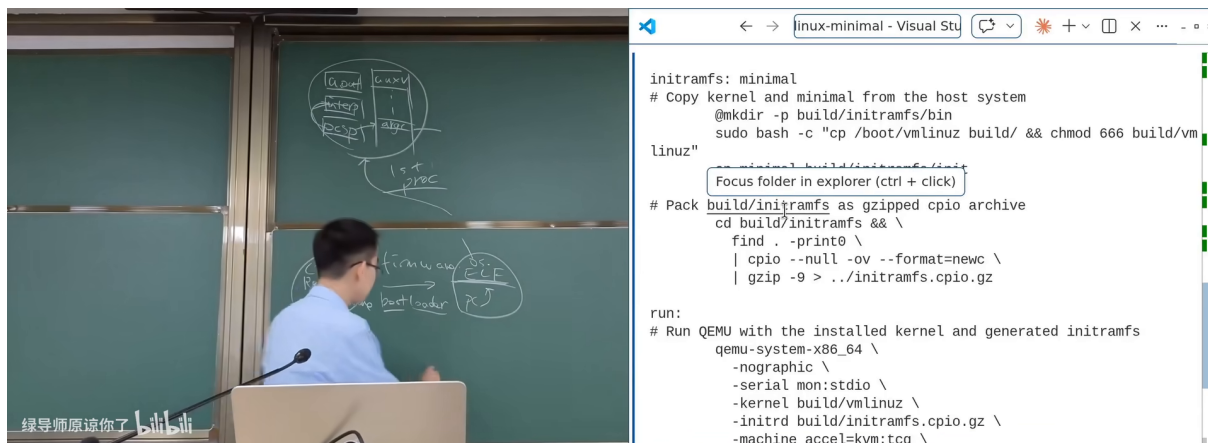


图 2: 类似结构的一条命令链: 拷贝 kernel/initrd、用 `cpio+gzip` 打包成 `cpio.gz`、QEMU 以 `-kernel/-initrd` 启动——这和后文我们手搓 minimal Linux 用的是同一套基础设施²

在这个 16-bit MINIX 中，`vi`、`cc`、`cat`、`grep`、`man`（且有颜色！）全部可用，所有 UNIX 工具链已经齐备。甚至连整套源代码都打包在 `/usr/src` 里——你可在 MINIX 里修改 MINIX 内核、编译后烧回磁盘重启。

2.2.2 微内核 vs 宏内核：一个过早的正确想法

Tanenbaum 想把 MINIX 设计成只有两个系统调用（`send/receive`）的微内核——所有文件系统、内存管理、进程调度都是用户态的服务进程，彼此用消息通信。

微内核为什么在 1987 年“输”了

技术上很优雅：服务可以崩溃而不影响内核，可以独立更新，权限更细。但物理上太慢——每一次 `read / write` 都要把参数打包成消息、做一次 IPC、等另一个进程调度上来处理、再回传一次消息，和 Linux 一条 `syscall` 指令几十个时钟周期就进内核相比，差了一两个数量级。2026 年的今天，硬件足够快、IPC 开销相对变小，以鸿蒙为代表的微内核重新进入主流——这是「合适的想法在不合适的时间」的典型。

² 视频画面时间区间：00:50:57–00:51:12；MINIX 的 `make minix2` 在讲述上承担同样的角色。

2.3 Linus Torvalds: just a hobby (1991-08-25)

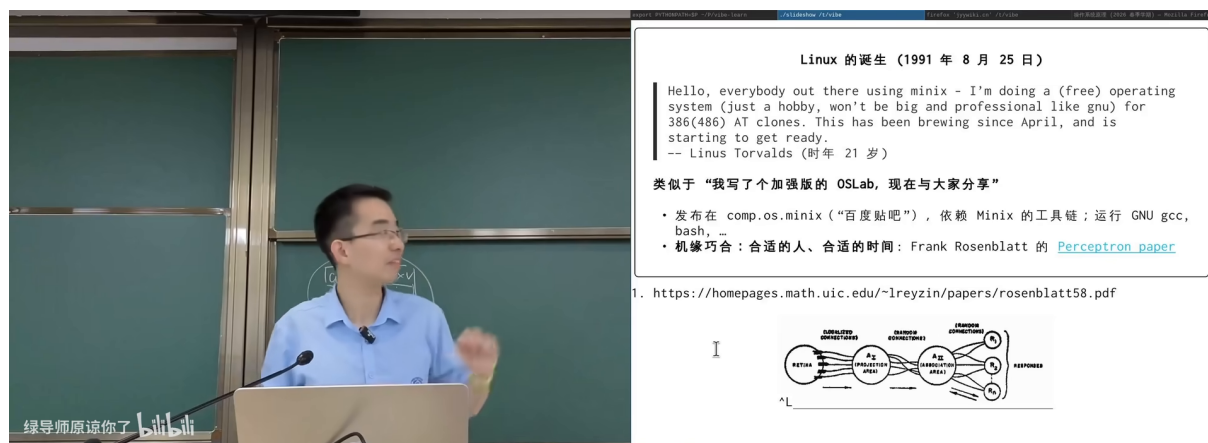


图 3: 1991 年 8 月 25 日, 21 岁的 Linus 在 comp.os.minix (“百度贴吧”) 上贴出了那封著名的邮件。图下方是 Frank Rosenblatt 的 perceptron 论文——讲者用来类比”合适的人, 合适的时间”³

邮件原文大意:

Hello everybody out there using minix —I’m doing a (free) operating system (just a hobby, won’t be big and professional like gnu) for 386(486) AT clones ...

Linus 当时手上有一台 386, 只想为自己写一个”能取代 MINIX 的、性能过得去的” UNIX-like 系统。早期 Linux 甚至还依赖 MINIX 文件系统——你得先把磁盘格式化成 MINIX 文件系统、上面跑 GNU 的 gcc/bash, 然后才能编译和运行 Linux。

为什么是芬兰、为什么是 Linus ?

讲者借这个故事引出一个观察: 芬兰冬天漫长寒冷, 年轻人只能在家取暖 + 写代码, 于是诞生了 Linus。David Diamond 的回忆录《Just for Fun: The Story of an Accidental Revolutionary》直言”**Revolution is not born, it just happens**”。对 2026 年的学生而言, 隐喻是: **你不需要等一个大型国家战略工程**——在 AI 时代, 从 0 到 0.1 的门槛前所未有地低, 你现在就有条件做自己的 Crazy OS。

2.3.1 时代的错位: Rosenblatt 的 perceptron

同一张幻灯片下方, 作者特意放了一张 Frank Rosenblatt 1958 年提出 perceptron 的论文。他指出: 感知机的神经元结构与今天深度学习的 neuron 几乎一模一样, 但 1958 年既没有算力也没有氛围——**合适的人提出了合适的东西, 却不在合适的时间**, 结果被 Minsky 的 XOR 悖论打下去, 等了 54 年才在 AlexNet 上复活。

³视频画面时间区间: 00:22:03–00:22:18。

2.4 Tanenbaum-Torvalds 大论战

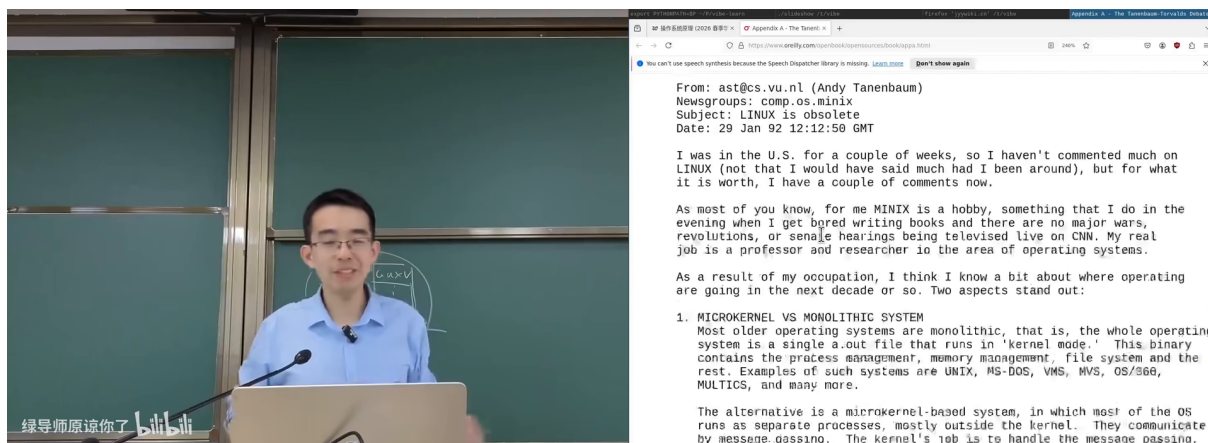


图 4: 1992 年 1 月 29 日, Andrew Tanenbaum 在 comp.os.minix 贴出 "LINUX is obsolete" 开喷。这是操作系统史上最出名的一次网络论战⁴

Tanenbaum 的核心观点:

- **MICROKERNEL vs MONOLITHIC SYSTEM:** monolithic 已经过时, MINIX 的微内核才是未来。
- 原文: "to me, writing a monolithic system in 1991 is a truly poor idea."——攻击性十足, 带着明显的"教授架子"。
- 结尾还不忘阴阳: "don't get me wrong, I am not unhappy with linux...but I would suggest that people who want a modern free OS look around"——然后推销自己 MINIX。

Linus 当然要喷回去; 更戏剧的是图灵奖得主 **Ken Thompson** 亲自下场参战。讲者对此感慨: 相当于"中科院院士在百度贴吧和大学教授 + 愣头青小伙子实名论战"——这在国内几乎不可想象, 但在那个时代的 usenet 上是常态。

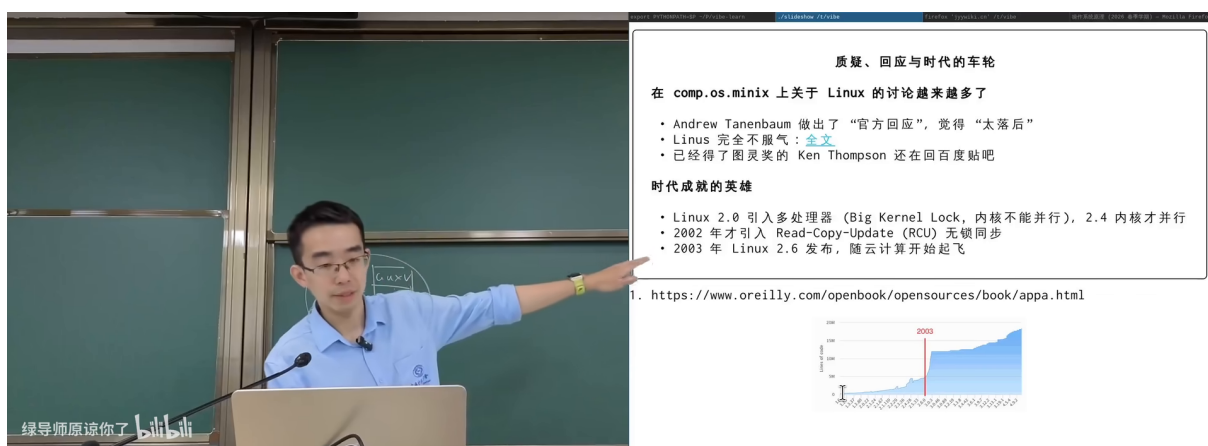


图 5: 质量、回复与时代的车轮: Linux 内核版本里程碑——2.0 引入 SMP、2.2 Big Kernel Lock、2.4 多处理器真正可并行、2.6 引入 Read-Copy-Update、2.6.x 后 USB 子系统大重构⁵

⁴视频画面时间区间: 00:31:03-00:31:18。

论战最终在代码里分出胜负——Linux 内核源代码量在 2.0 之后经历了断崖式增长和好几次核心子系统的推倒重来，逐渐从「hobby project」变成 IBM 等大厂入场的严肃服务器系统，再到 2003 年前后恰逢 Google 等云计算基建起飞，Linux 一举成为事实标准。现在它已经**既不是纯宏内核也不是纯微内核**——一些模块化与服务化特性都被吸收进来。

2.4.1 名场面：Linux vs NVIDIA (2012)

2012 年 AlexNet 引爆深度学习的同一时期，Linus 在一次公开采访中指着 NVIDIA 的方向竖起中指，称：

NVIDIA is the single worst company we've ever dealt with.

十四年后的今天，NVIDIA 已经是地球上市值最高的公司。而 Linux kernel 邮件列表被永久归档——**说过的每一句话都能被翻出来**。这也是讲者顺势推出的一条职业建议：在公开场合发言，要对得起十年后被 Google 搜到的自己。

2.5 中场插曲：AI 时代的学习方式

讲到这里 jyy 岔开话题分享他自己现在的学习流程：

1. 把一篇 paper（甚至 100 多页的 PhD 论文 PDF）扔到 iCloud 共享目录；
2. 后台用 Claude Code + Coding Plan 自动跑 OCR + 多层 summary（1 句话 / 1 段 / 1 页纸）；
3. 在 Claude Code 终端里对话式追问细节；
4. 对话历史作为 memory 被持久化，下次接着用。

Caveman Skill 的有趣之处

Caveman 是一个流行的 Claude Code skill：它用**极短、语法不通的 prompt**（甚至文言文）去压缩输出以节省 token。原理是**利用 agent 的容错性**——压缩确实会降低单次准确率，但因为 agent 有自主性，它发现失败会自己重试，整体 overall 成功率变化不大，token 却省下来了。讲者自己的 CLAUDE.md 里也有类似一条：“用中文、尽量短”——原理相同。

从 0 到 0.1 的革命

AI 时代一个最深刻的变化是：「从 0 到 0.1」前所未有地容易。过去要一个天才少年独自撑满几年才能做出的原型，今天 bootstrap 一个 agent 就能出一个能跑的 demo。这对学生的意义是：**你的“足够偏门的想法”就是今天最宝贵的资源**，不要等被“国家战略”安排去做，而是自己去“百度贴吧发帖子”。

2.6 本章小结

历史线传达两条信号：

- 大的系统都起源于非常小的东西（27 行汇编 fork、一封邮件、一本教科书）；
- 创新并非国家规划的产物——它依赖于**合适的时间、合适的地点与合适的人**的偶然相遇。

⁵视频画面时间区间：00:34:03–00:34:18。

今天的学生处在一个更好的时代：你们不缺算力、不缺模型，甚至不缺「crazy OS」这样的课——唯一需要的就是**把一个偏门的想法真正做出来**。

3 从 initramfs 开始构建 Linux 世界

这一章是全课的技术高潮。它回答一个非常朴素的问题:Linux 从电源键按下,到你看到 `systemd` 成为 1 号进程,中间到底发生了什么?

3.1 起点:所有系统的”初始状态”

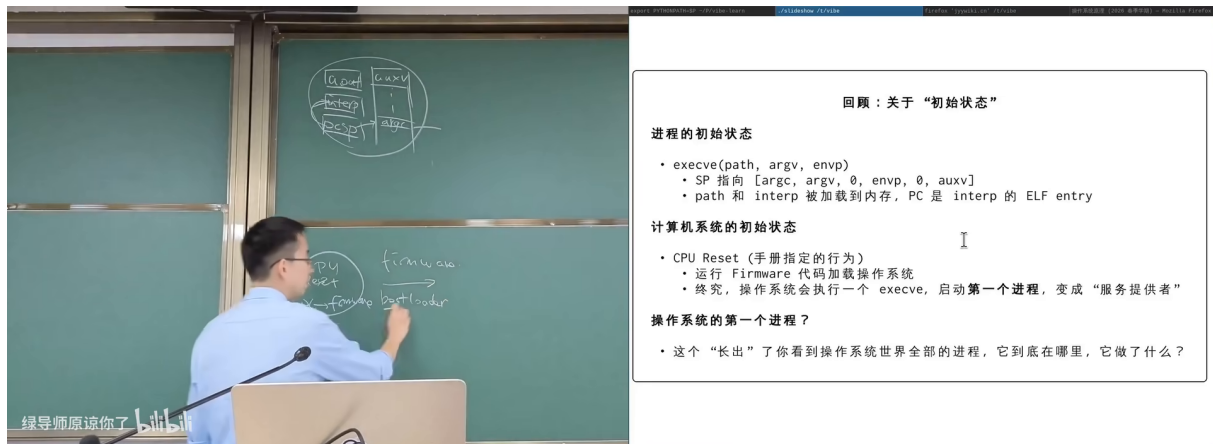


图 6: 初始状态的三层类比: 进程的初始状态由 `execve` 决定; 计算机系统的初始状态由 CPU Reset 决定; 而操作系统的”第一个进程”呢? 这就是本章要回答的问题⁶

三个层次的”初始状态”可以这样对齐起来:

- **进程的初始状态:** `execve(path, argv, envp)` 之后, SP 指向初始栈 (`argc+argv+envp+auxv`), PC 指向 interpreter/ELF entry。
- **计算机系统的初始状态:** CPU Reset 后寄存器和内存都在预定义状态, PC 指向 firmware 代码。
- **操作系统的”第一个进程”:** 操作系统加载完成后必须 `execve` 某个用户态程序, 之后它自己就退居幕后只处理 syscall 与中断。

3.1.1 启动键的全景

1. 按下电源键 → CPU Reset → PC 指向 firmware;
2. Firmware (BIOS/UEFI/U-Boot) → 启动加载器 (bootloader);
3. Bootloader 把 OS 的 ELF (一个静态连接的大 ELF) 加载进内存, 跳入其第一条指令;
4. 内核初始化 (MMU、调度器、文件系统、驱动...) 结束后, **`execve` 第一个用户态进程;**
5. 之后操作系统变成”服务提供者”: 系统调用由进程发起、内核响应; 后台同时处理中断、刷盘、抢占调度等。

那个”第一个进程”到底是谁? ps 告诉你所有进程的根是 `systemd`——但 这其实是一个巧妙的误导, 真相在下一节揭晓。

⁶ 视频画面时间区间: 00:47:12–00:47:27。

3.2 initramfs: 第一个进程住在哪里？

initramfs

initramfs = "Initial RAM File System", 断句是 **init-RAM-FS**。内核刚加载完时**还不知道磁盘在哪**——不知道文件系统在 USB、SATA 还是 NVMe 上, 也不知道对应的驱动是否已经被编进内核。它需要一个**不依赖任何外设**的根文件系统, 用来:

1. 加载磁盘驱动;
2. 找到真正的根分区;
3. 挂上真正的根文件系统;
4. 用 `pivot_root` 切换过去。

这个文件系统就是 **initramfs**, 由内核的 `bootloader` 参数 `-initrd` 提供, 格式是一个 **cpio.gz** 归档。

为什么 apt upgrade 里 update-initramfs 最慢？

如果你在 Debian/Ubuntu 上升级内核或某个关键模块, 末尾几乎总会停在 `update-initramfs` 这一步上。原因是 **initramfs** 是提前**烘焙**好的、随内核一起加载的 `cpio.gz`——里面嵌入了当前内核的驱动模块副本, 更新内核意味着要重新打包这个归档(扫文件、算依赖、压缩)。

3.3 动手: 最小的 init

3.3.1 第一步: 一个只会 write+exit 的 x86-64 静态 ELF

讲者先给出一个只依赖两条系统调用的最小二进制 `minimal`:

```
1 # 打印一行字, 然后立刻退出
2 write(1, "This is a minimal binary\n", 25);
3 exit(0);
```

Listing 1: 最小 init 的基本行为

它是静态连接的 x86-64 ELF——即使在 ARM 树莓派上也能运行, 因为现代 Linux 会通过 `binfmt_misc` + QEMU user mode 自动把陌生架构的二进制路由到模拟器, 用户态指令由 QEMU 模拟、系统调用直接转给宿主内核。

3.3.2 第二步：把它打包进 initramfs

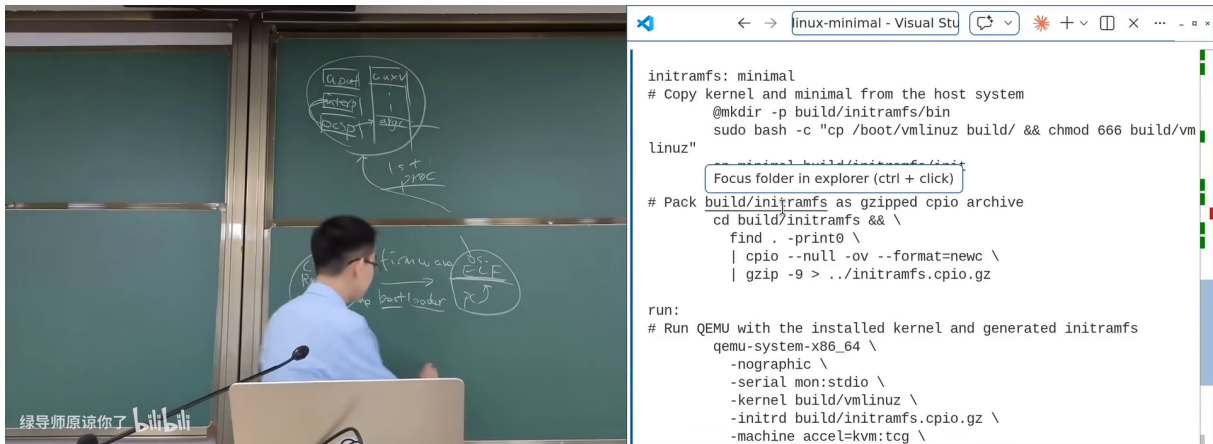


图 7: Makefile 的三条规则: (1) 从宿主 build 目录拷 kernel + initramfs 根目录; (2) 用 `find | cpio --format=newc | gzip` 打包成 `initramfs.cpio.gz`; (3) 用 `qemu-system-x86_64 -kernel ... -initrd ...` 启动⁷

关键参数:

- `-kernel build/vmlinuz`: 要加载的 Linux 内核镜像;
- `-initrd build/initramfs.cpio.gz`: 打包好的根文件系统;
- `-append "rdinit=/init"`: 通过内核命令行显式指定 1 号进程的路径——覆盖默认的 `/sbin/init`, `/etc/init`, `/bin/init` 查找顺序;
- `-nographic`: 不开 QEMU 图形窗口, 用串口做 console。

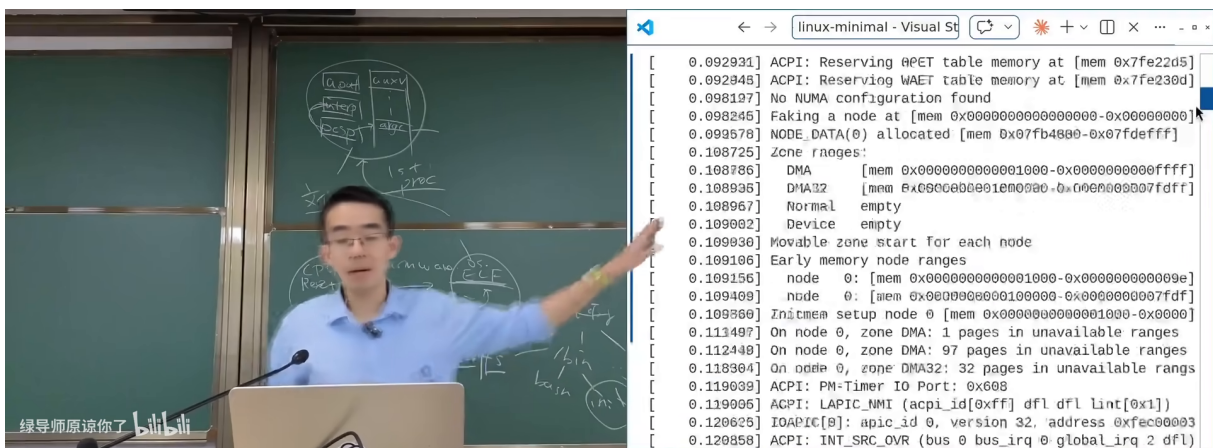


图 8: 从 QEMU 串口看到的真实内核启动日志: ACPI 扫表、内存分区 (DMA/DMA32/ Normal)、memblock 分配器、SMP 配置、初始化 PIC/APIC...你在发行版开机时一闪而过的那些消息, 在 `-nographic` 模式下是可以一行一行读的⁸

⁷ 视频画面时间区间: 00:50:57-00:51:12。

⁸ 视频画面时间区间: 00:54:57-00:55:12。

The image is a composite of two parts. On the left, a man in a blue shirt is pointing at a chalkboard. The chalkboard has two diagrams. The top diagram is a circular memory layout divided into sections labeled 'data', 'code', 'bss', and 'rodata'. Arrows indicate the flow of data between these sections and a 'main' function. The bottom diagram is a flowchart showing the execution of 'firmware' and 'init' processes, with labels like 'init/romfs' and 'bin'.

On the right, a terminal window titled 'linux-minimal - Visual St' displays the output of the command 'cat /proc/kallsyms'. The output lists various kernel symbols and their addresses, including 'evm: security.capability', 'tsc: Refined TSC clocksource calibration: 1000.000 z', 'clocksource: tsc: mask: 0xffffffffffffffff max_cycles', 'clocksource: Switched to clocksource tsc', 'clk: Disabling unused clocks', 'Freeing unused decrypted memory: 2036K', 'Freeing unused kernel image (initmem) memory: 2804K', 'Write protecting the kernel read-only data: 26624k', 'Freeing unused kernel image (text/rodata gap) memork', 'Freeing unused kernel image (rodata/data gap) memork', 'x86/mm: Checked W-X mappings: passed, no W-X pages.', 'Run /init as init process', 'This is a minimal binary.', 'Kernel panic - not syncing: Attempted to kill init!', 'CPU: 0 PID: 1 Comm: init Not tainted 6.1.0-32-amd64', 'Hardware name: QEMU Standard PC (i440FX + PIIX, 1994', 'Call Trace:', '<TASK>', 'dump_stack_lvl+0x44/0x5c', 'panic+0x118/0x2f4', 'do_exit.cold+0x15/0x15'.

为什么杀 `init` 会让整个操作系统罢工？

1 号进程是所有孤儿进程的**最终收养人**，也是内核与用户世界唯一的接口。如果 1 号进程死了，内核没有办法“优雅降级”——只能立即 panic。这不是 bug，而是内核主动维持“用户态世界必须存在”这一不变式。对比一下：普通进程 `exit`，内核只是回收资源；只有 `init` `exit` 才会触发整机 panic。这解释了为什么平时看到的 `systemd` 即使在 `Ctrl+C` 洪流中也不会挂掉——它被特殊对待。

3.4 Linux 真正的启动全流程: initramfs + pivot_root

接着 jyy 把宿主树莓派真正的 `/boot/initrd.img` 喂给 Claude Code 解包。一个关键的观察：

AI 会翻车，但它会回去自己改

AI 第一次把 `initrd` 解成 7.3 MB 的目录，但原文件是 23 MB。**你只要察觉到这个体积不一致，就可以追问**”肯定还有东西没解”。AI 发现 `cpio trailer` 之后还有 19 MB 额外数据，自己写脚本去扫；最终找到第二段 `zstd` 压缩的 `cpio` 并解出完整目录树。

这一节不是在讲 `initrd` 细节，而是在讲 **coding agent 真正强大的地方**——它像人一样探索、遇到矛盾会自己回头修。你作为使用者唯一不能被替代的能力是 **对”观测 / 预期不一致”敏感**。

- 核心启动脚本 `/init` (shell 脚本, 不是 ELF);

13

- BusyBox 提供的数百个命令；
- 各种文件系统的驱动模块 (ntfs-3g、ext4、vfat...);
- 键盘布局、字体配置、启动画面；
- 网卡、显示、USB 的 firmware blob。

为什么磁盘挂不上时会掉进一个命令行？

经常“把系统搞坏”的同学应该见过——开机后 Linux 停在一个看起来非常朴素的 BusyBox shell 上。原因是内核已经把 **initramfs** 挂好了，但在 initramfs 脚本尝试找你真实根分区时失败（磁盘坏了、fstab 写错、加密盘没解锁...），于是脚本 fallback 给你一个 shell。此时内核其实好好的，你可以在这里手动 **fsck**、**mount** 别的盘。

3.4.2 pivot_root: 从 initramfs 换到真实根

这是全讲最关键的”一颗螺丝钉”——它才是 PID 1 不是 systemd 的原因。

pivot_root 的行为

`pivot_root(new_root, put_old)` 是一个系统调用，它交换调用进程的根挂载和 `new_root`——调用之后，所有新 `execve` 的绝对路径都相对于 `new_root` 解析。initramfs 脚本在最后做的事情就是：

1. `mount` 真实根分区到 `/new_root`;
2. `pivot_root /new_root /new_root/old`;
3. `exec /sbin/init` (才是我们熟悉的 systemd!)

systemd 其实不是 Linux 启动时的第一个进程——它是 PID 1，但在它之前，initramfs 里有一个叫 `/init` 的 shell 脚本才是最初的 PID 1。pivot_root 之后 `exec` 保留了 PID，所以它看起来像是”一直都是 systemd”。

3.4.3 一份自制的完整 init 脚本

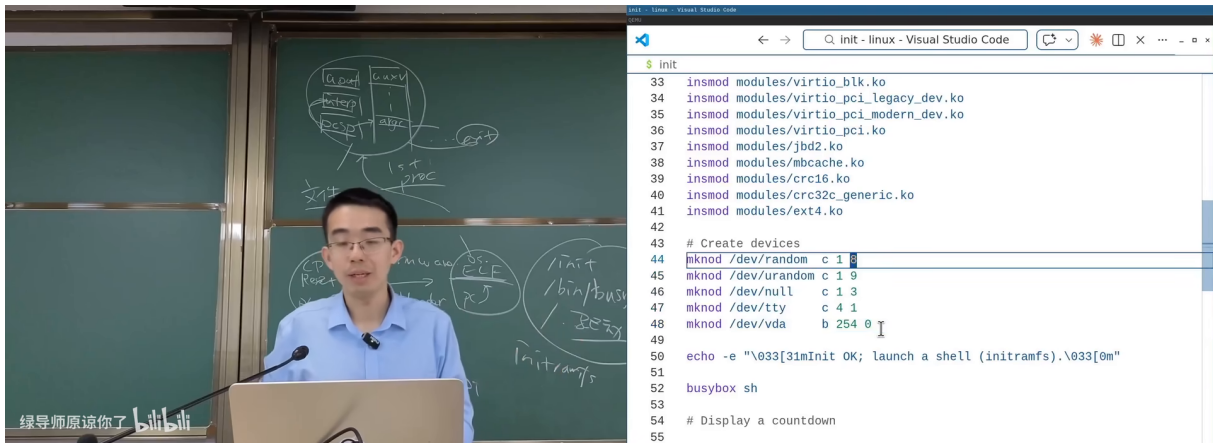


图 10: 作者自己写的第二份 init: 先 insmod 一堆驱动 (virtio-blk、virtio-net、virtio-pci...), 然后 mknod 建 /dev/random、/dev/null、/dev/vda..., 最后启动一个 BusyBox shell, 给 3 秒倒计时后 pivot_root 到真实根¹⁰

完整流程可以用一段 shell 伪代码概括:

```
1  #!/bin/busybox sh
2  # 1. 把 BusyBox 展开成所有命令的符号链接
3  for cmd in $(/bin/busybox --list); do
4      /bin/busybox ln -s /bin/busybox /bin/$cmd
5  done
6
7  # 2. 创建 procfs、sysfs、devfs 等内核对象的挂载点
8  mkdir -p /proc /sys /dev
9  mount -t proc proc /proc
10 mount -t sysfs sys /sys
11
12 # 3. 加载磁盘和网卡驱动
13 for mod in virtio_pci virtio_blk virtio_net 8139cp; do
14     insmod /lib/modules/$mod.ko
15 done
16
17 # 4. 创建设备节点 (老派做法; 真实系统用 udev/mdev)
18 mknod /dev/vda b 254 0
19 mknod /dev/tty1 c 4 1
20 # ...
21
22 # 5. 跑一个可交互 shell 让用户把玩
23 echo "init OK — launching shell (initramfs)"
24 busybox sh
25
26 # 6. shell 退出后, 挂载真实根并 pivot_root
27 sleep 3; echo "3"; sleep 1; echo "2"; sleep 1; echo "1"
28 mount -t ext4 /dev/vda /newroot
```

¹⁰视频画面时间区间: 01:12:42–01:12:57。

29 `exec switch_root /newroot /init # 相当于 pivot_root + exec`

Listing 2: 手搓 minimal Linux 的 init.sh 脉络

3.5 pivot_root 之后：一个真正可用的 Linux

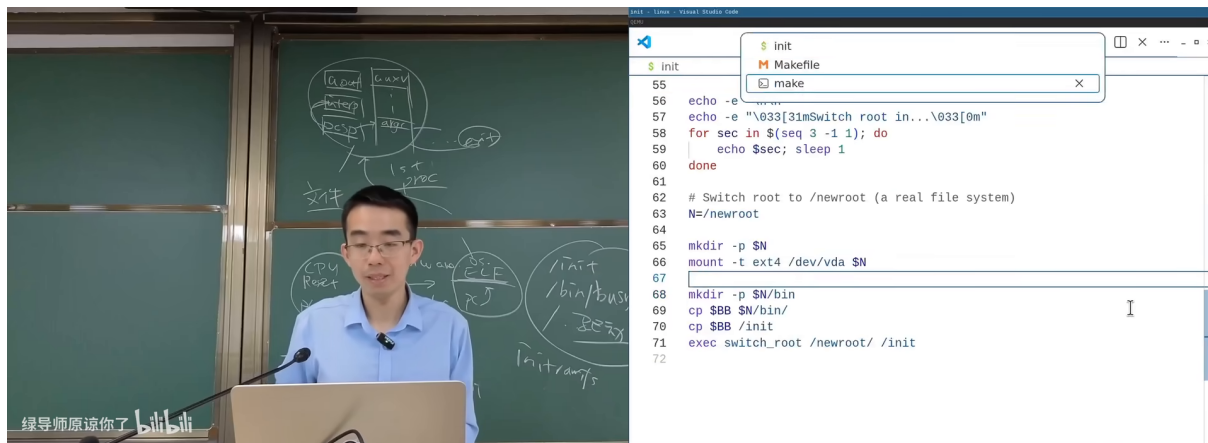


图 11: pivot_root 之后作者打印“jyy’s minimal Linux” 红字；文件系统变成了磁盘上的 ext4；能启动 BusyBox 的 httpd -p 8080，外面宿主机通过 QEMU 端口转发用浏览器访问 localhost:8080/init ——真的看到了虚拟机里的 init 脚本内容¹¹

课程高潮：从零到一个联网的 Linux

从一个空的 initramfs 起步，经过：

1. BusyBox + 符号链接长出命令行工具集；
2. insmod 加载块设备 / 网卡驱动；
3. mknod 创建 /dev 对象；
4. mount -t proc/sysfs 挂起内核对象视图；
5. pivot_root 切到真实根；
6. 第二段 init 再做一遍上面的事；
7. ifconfig + httpd -p 8080 起网络服务；
8. QEMU 端口转发让宿主机浏览器真正读到虚拟机里的文件。

整个过程完全没有”魔法”。每一步都是一个或几个系统调用——mount、mknod、execve、fork、socket、bind、accept、read、write。操作系统的全部本领就是一组对象 + 一组 syscall。

¹¹ 视频画面时间区间：01:18:27–01:18:42。

3.6 从系统调用到应用生态

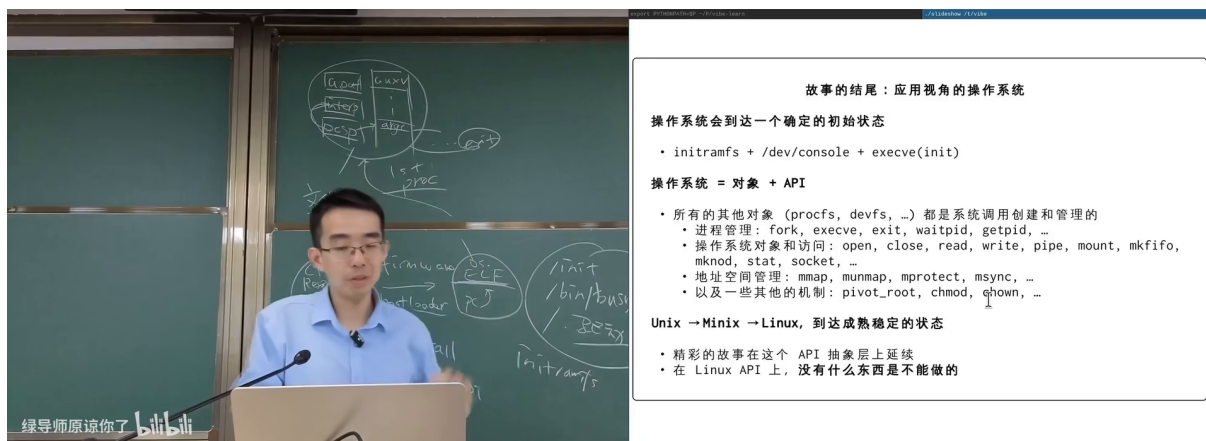


图 12: 故事的结局：从 `initramfs + /dev/console + execve(init)` 开始，操作系统给应用世界提供的就两样东西——**对象** (`procfs`, `devfs`、各种 `mount-able` 抽象) 与 **API** (`fork`, `execve`, `read`, `write`, `waitpid`, `mmap`, `munmap`, `close`, `socket`, `stat`, `send`...) ¹²

狭义的 OS 就是这么一张 API 表。广义的 OS 还包括：

1. **基本运行库**：libc、libm、libstdc++、OpenGL / Vulkan、OpenSSL...
2. **核心工具链**：gcc / clang、binutils、make / cmake、git...
3. **包管理 / 应用分发**：apt (1998)、rpm、App Store (2008)，到今天的 npm / pip / Hugging Face / Ollama。
4. **辅助工具**：安装、版本、系统管理工具。

为什么「中国没有国产操作系统」其实是伪命题？

真正难的从来不是 Linux kernel 那一两千万行——相比一个复杂业务系统（可能上千万行），kernel 反而是**最小的那一层**。难的是上面的**整套生态**：核心工具链、基本运行库、包管理、开发者社区、第三方软件。Debian 这类项目用了二十多年才把一整套开源供应链 (`unstable` → `testing` → `stable`) 打磨到今天的状态；NPM / Hugging Face 的内容分发网络也是整个互联网行业十几年迭代出来的。这不是一个命令“`apt install 国产操作系统`”可以复刻的。

3.7 本章小结

技术闭环：

- **启动链**：Reset → Firmware → Bootloader → Kernel → `initramfs/init` → `pivot_root` → 真实根 → `systemd` (仍是 PID 1)。
- **第一进程可控**：通过自制 `initramfs + rdinit=` 内核命令行，可以把任何 ELF / 脚本塞成 PID 1。

¹² 视频画面时间区间：01:21:12–01:21:27。

- **「没有魔法」**：所有后续行为都是系统调用的排列组合；BusyBox 是最小化演示，Debian 是工业化演示。
- **应用生态 » 内核**：一张 API 表不等于一个操作系统；真正让 Linux 无处不在的是围绕它构建的那整座开源供应链。

4 总结与延伸

4.1 讲者核心观点

1. **整个虚拟化部分其实只在讲一件事**——「操作系统给应用程序提供的是什么样的 API」。前面所有课程都是在教你怎么读 man pages、怎么追 strace、怎么在 AI 帮助下把任何一个系统调用拆到字节级。
2. **OS 的第一进程可控**。通过 `initramfs + rdinit=`，你可以把任何一个自己写的 ELF 塞到 Linux 启动链的最顶端。那些启动慢、启动快、磁盘挂不上、`systemd` 卡住之类的日常现象，一次就能理解透。
3. **`systemd` 是 PID 1 但不是真正的第一个进程**。真正的第一个进程是 `initramfs` 里的 `/init` shell 脚本，`pivot_root + exec` 之后 PID 被继承下来。
4. **「没有魔法」原则贯穿整门课**。从打开抖音到内核启动，每一个像素点背后都是一串系统调用。
5. **AI 工具已经把从 0 到 0.1 的门槛夷平**。你缺的不是聪明和资源，而是一个“偏门但可做”的想法 + 真的去做的勇气。
6. **真正难的操作系统工作是生态，不是内核**——这才是为什么 Debian、RPM、APT 这些工具花了二三十年才演化到今天的样子。

4.2 可操作的知识蒸馏

如果这节课只留下三条可直接检验自己掌握情况的东西：

自测三问

1. 在你的发行版上运行 `lsinitramfs /boot/initrd.img-$(uname -r) | head`，能不能解释看到的每一个目录？
2. 能不能手写一个 `initramfs (cpio.gz)`，让 QEMU 启动到一个只打印“hello”然后 `sleep` 的静态 ELF？
3. 能不能用 `strace` 观察 `systemd` 启动过程中的前 10 个系统调用，并说出每个调用在“构建世界”这张地图上的位置？

4.3 跨课程的 synthesis

- 与第 11 讲（链接和加载）：第 11 讲解释了“一个进程的初始状态从哪来”（ELF loader + `auxv`）；第 12 讲把尺度扩大到“一个系统的初始状态从哪来”（firmware + `initramfs`）。两节课是同一道问题在两个粒度上的展开。
- 与第 5 7 讲（进程、`mmap`、`fd`）：这里所有的 `fork`、`execve`、`mmap`、`mount`、`mknod` 都是那几节课引入的抽象。`initramfs` 只是把它们从“概念题”变成了“现实可见的串联”。
- 与即将到来的并发部分（13 讲起）：讲者在本讲提到，整个虚拟化部分只处理“单进程视角下的操作系统 API”；下一讲 多处理器编程：从入门到放弃会开始引入并发带来的更大麻烦——同样是系统调用，但多核执行序会让原本“确定性”的状态变成一片乱麻。

4.4 拓展阅读

- Dennis Ritchie, *The Evolution of the Unix Time-Sharing System*, Bell Labs Technical Journal, 1984.
- Andrew S. Tanenbaum, *Operating Systems: Design and Implementation* (即 MINIX book) , Prentice Hall, 1987.
- David Diamond & Linus Torvalds, *Just for Fun: The Story of an Accidental Revolutionary*, HarperBusiness, 2001.
- Frank Rosenblatt, *The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain*, 1958. <https://homepages.math.uic.edu/~lreyzin/papers/rosenblatt58.pdf>
- comp.os.minix 上 Tanenbaum vs Torvalds 的原始论战存档: <https://www.oreilly.com/openbook/opensources/book/appa.html>
- MINIX 3 项目主页: <https://minix3.org/>
- Linux 内核源码中 `init/main.c` ——从 `start_kernel` 到 `run_init_process` 的那条路径, 是本讲所有事情的内核侧 counterpart。
- `man 8 initramfs-tools`、`man 2 pivot_root`、`man 7 busybox`。